

Mini Guardian: A Non-Intrusive Userspace Native Security Architecture

Preserving Userspace Data and Workflows in Bounded Compute Environments

Technical Paper v0.6

Jayce Automata Research

July 2026

Abstract.— We present Mini Guardian, an advanced userspace security sidecar application deployed natively as a systemd service on Linux hosts. Mini Guardian wraps the Jayce Kernel substrate to provide defense-in-depth protection for host systems without modifying their kernel, partitions, or userspace files. It integrates eBPF/XDP network filtering, Landlock filesystem sandboxing, and chained seccomp-BPF filters. It introduces a tiered “Brain” and “Memory” architecture that aggregates EWMA behavioral anomalies, cross-component correlations, and autonomous feedback loops into a local-only threat graph with in-memory attack memory. For multi-node deployments, Mini Guardian implements a gossip-based peer-to-peer cluster with mTLS transport, HMAC-SHA256 integrity, and hash-chained anti-replay. Defense of the daemon itself covers four fronts: release hardening (stripped/LTO binary, statically linked via musl, immutable on disk), **string obfuscation** of capability-revealing paths, **anti-analysis** detection of reverse-engineering tools targeting the daemon, and a **separate antifreeze sentinel process** that survives SIGSTOP attacks against the daemon. A ship-seal phase closes the loop: sealed-hash cross-verification between daemon and sentinel, lethal response to tracing, argv/envIRON scrubbing, read-only deployment, and surfaced host attestation (kernel lockdown / Secure Boot / TPM). A shadow-backup layer auto-restores tampered security-critical files while preserving attacker evidence, and a container monitor maps the container threat surface. We describe the threat model, deployment topology, security mechanisms, and competitive assessment, demonstrating an overall security score of 8.0/10 compared to CrowdStrike Falcon’s 6.9/10.

1. Introduction

Modern security systems often present a false dichotomy: operators must choose between deep, intrusive kernel-level isolation (e.g., custom microkernels, hypervisors, or custom OS builds) that wipes existing filesystems and breaks user workflows, or weak userspace agents (e.g., traditional EDR/SIEM agents) that offer visibility but fail to prevent exploits or containment in real-time.

Mini Guardian bridges this gap by introducing a **non-intrusive native deployment model**. Operating entirely in userspace as a systemd service with a companion CLI (`miniguard-status`), it provides a deterministic, capability-gated security wrapper around existing systems. It enforces strict sandboxing, network filtering, and behavioral correlation while leaving the host OS kernel, disk partitioning, and userspace data directories intact.

1.1 Core Architecture Principles

Mini Guardian operates under four non-negotiable architectural guidelines:

- **Zero Intrusion:** It does not require host kernel modifications, driver injections, or custom kernel module compilations.
- **Data Preservation:** It maintains 100% of host userspace files and databases, sandboxing processes at logical boundaries rather than physical disk sectors.

- **Ambient Host Isolation (No Ambient Trust):** It treats the host OS environment (environment variables, config files, /proc status) as untrusted input, verifying all configurations through signed profiles or kernel-mediated shims.
- **Local-Only Auditing:** Telemetry, logs, and state-machine historical states are compressed and written locally via a custom JVC (Jayce Vector Compression) binary format, eliminating external cloud data-leaks. Optional gossip-based cluster sharing is cryptographically authenticated and explicitly opt-in.

1.2 The Deterministic Tick (System Heartbeat)

Unlike traditional security agents that run asynchronously and depend on host operating system clocks, Mini Guardian relies on the simulated Jayce Kernel’s **1,000 Hz logical tick** as the system’s heartbeat:

- **Temporal Independence:** Host-level wall clocks are subject to user tampering, clock-drift exploits, or debugger pausing. By using logical tick increments as the primary axis of time, Mini Guardian remains independent of host-level temporal alterations.
- **Deterministic Execution:** Each tick triggers a sequential run of the security state machine. Code evaluation, telemetry snapshot capture, and verification checks happen in a non-preemptible, pre-defined sequence.
- **Continuous Invariant Auditing:** Algorithms (such as the Predictive Execution Horizon and Impossible State Detector) process data at these precise tick boundaries. This ensures timing quantile checks, counter regressions, and hazard forecasts are computed against a predictable, heartbeat-driven telemetry baseline.

2. System Architecture and Crate Taxonomy

Mini Guardian is implemented as a Rust workspace containing four key crates that run in userspace:

Crate	Responsibility
miniguard	Host-side monitoring daemon (systemd service), eBPF program management, cgroup sandboxing, gossip cluster, autonomous Brain with 5 feedback loops
miniguard-status	CLI utility for querying agent health, blocklists, security posture, and competitive assessment comparison
trust_core	Shared library containing the Prompt Guard (24 attack families, 200+ patterns), Memory Poisoning Defense (5-layer AI pipeline protection), and Semantic Analyzer (32-dim static embedding model)
jayce_kernel	Std-compatible simulator of the bare-metal Jayce Kernel state-machine, providing the 1,000 Hz deterministic tick loop

Table 1: Mini Guardian Crate Division

The daemon (miniguard) runs as a native systemd service deployed at /usr/local/bin/miniguard. It loads its configuration from /etc/miniguard/miniguard.toml (TOML format). It wraps the simulated kernel runtime, mapping OS-level signals (exec events, network connections, file changes) to the simulated capability broker, and feeds state transitions back to the host via automated incident response. A companion CLI (miniguard-status) provides real-time agent health, blocklist inspection, configuration profiles, and a full competitive security assessment scored against industry benchmarks.

2.1 Native Deployment Hardware Module Applicability

Mini Guardian deploys natively on Linux (Fedora 44+, Ubuntu 24.04+, Debian 12+) and operates entirely in userspace. Since it does not require Docker, container runtimes, or kernel module compilation,

it introduces zero additional attack surface from container escape vectors. The native deployment model enables full access to:

- **eBPF/XDP programs:** Network filtering at the NIC driver level via XDP
- **Landlock LSM:** Filesystem sandboxing without kernel configuration
- **Chained seccomp-BPF filters:** Self-sandboxing of the daemon process
- **Systemd integration:** Automatic restart on failure, resource limits, journald logging

However, raw hardware-bus operations (USB controller access, RF monitor mode) remain out of scope for the userspace sidecar, matching the same division described in the Jayce Kernel specification.

3. Sandboxing and Containment (The Shroud)

Mini Guardian establishes a series of defense-in-depth isolation layers on the host system without requiring administrative configuration changes.

3.1 Chained Seccomp-BPF Filters

To secure the security daemon itself, Mini Guardian applies self-sandboxing. Due to the 255-instruction limit in standard BPF filters, it chains two separate programs:

- **Program 1 (Low-Range):** Filters syscalls 0–255. It blocks dangerous operations such as anti-debugging (`ptrace`, `process_vm_readv`), namespace escapes (`chroot`, `setns`), and kernel module modification (`init_module`, `delete_module`).
- **Program 2 (High-Range):** Filters syscalls 256+. It blocks modern kernel exploit surfaces (such as `io_uring_setup` and `io_uring_enter`).

3.2 Landlock LSM Sandboxing

Using Linux’s Landlock Security Module, Mini Guardian enables processes to restrict their own file system access. It maps these into default, strict, and paranoid profiles, ensuring that even if an application is compromised, the kernel prevents it from writing to directories outside its capability envelope (e.g., `/boot`, `/etc`, `/usr`).

3.3 eBPF and XDP Network Enforcement

Mini Guardian loads BPF programs to filter host-level TCP/IP operations:

- **XDP NIC Filters:** Drops malicious packets from blocked IP addresses directly at the network interface card driver level, bypassing the host IP stack entirely and mitigating DDoS pressures.
- **XDP DNS Inspection:** Analyzes DNS query label counts and entropy to intercept DNS tunneling and exfiltration attempts.
- **Tracepoint Monitors:** Traces `execve`, `openat`, and `memfd_create` events to detect in-memory execution.

4. The Brain and Memory System

To detect and contain complex threat vectors, Mini Guardian relies on a tiered state and recall engine.

4.1 The Guardian Brain: Autonomous Intelligence with 5 Feedback Loops

The Brain (`brain.rs`) is Mini Guardian’s autonomous learning subsystem. It runs entirely locally with no cloud dependency, using five parallel feedback loops to detect, learn from, and respond to threats:

1. **EWMA Anomaly Loop:** Exponentially weighted moving average baselines for 8 metrics per process (cloning frequency, file modifications, capability probes, network flows, latency, scheduling jitter, memory access patterns, and IPC rates). Z-score deviations are computed before folding observations into the baseline, preventing slow-rolling contamination.
2. **Attack Memory Loop:** Persists recognized attack patterns to disk in JVC binary format with HMAC-SHA256 integrity verification. Patterns survive daemon restarts and can be shared across cluster nodes via gossip. The attack memory stores up to 1,024 signatures with EWMA-derived confidence scores.
3. **Automata State Loop:** Tracks the daemon's security posture through five deterministic states:
Normal (Calm) -> Suspicious -> Hostile -> Paranoid -> Recovery
State transitions are gated by configurable thresholds, with each state triggering progressively stricter containment measures (output jittering in Suspicious, NIC-level IP blocking in Hostile, full Landlock maximum lockdown in Paranoid, cooldown window in Recovery).
4. **Cross-Component Correlation Loop:** Aggregates signals (IPs, ASNs, domains, syscall sequences) across separate modules. When a single threat signature appears in multiple components, the correlation engine escalates the severity level, enabling detection of coordinated multi-vector attacks.
5. **Health Feedback Loop:** Monitors the daemon's own resource consumption (CPU, memory, file descriptors, tick timing) and adjusts scan aggressiveness and polling rates to prevent resource exhaustion on the host system.

All five loops operate on the deterministic 1,000 Hz tick, ensuring predictable scheduling and bounded resource consumption.

4.2 Tiered Memory Graph Architecture

Mini Guardian implements a tiered storage architecture for threat logs and security events (`memory_graph.rs`), avoiding memory exhaustion on resource-constrained hosts while maintaining quick access to active threat signatures:

- **Hot Tier:** Backed by an in-memory VecDeque of raw 64-byte ThreatSignature records. Designed for $O(1)$ ingestion and pattern matching with LRU eviction.
- **Warm Tier:** Compressed binary segments on disk using JVC XOR-delta + RLE. When the Hot Tier exceeds its elastic budget, entries are evicted, compressed, and flushed to warm segments (`warm_N.jvc`).
- **Cold Tier:** Compacted history periodically archived in `cold.jvc`. Full HMAC-verified archive with versioned binary wire format.

4.3 JVC (Jayce Vector Compression) Codec

Mini Guardian uses a custom **JVC (Jayce Vector Compression)** binary wire format to store threat signature vectors. The format uses a 4-byte magic (`JVC\x01`), version field, mode byte, and entry table with length-prefixed entries. For ThreatSignature arrays, it employs **XOR-Delta + Run-Length Encoding (RLE)**:

- **XOR-Delta Transformation:** For each signature record S_t , we calculate the bitwise difference against the preceding record S_{t-1} :

$$\Delta_t = S_t \oplus S_{t-1}$$

- **Run-Length Encoding (RLE):**

- If $\Delta_t = 0$ (meaning the current signature is identical to the previous one), the codec increments a 1-byte repetition counter. If the counter reaches 255 or a difference occurs, it writes a 0x02 tag byte followed by the counter value.
- If $\Delta_t \neq 0$, the codec writes a 0x01 tag byte followed by the raw 64-byte Δ_t array.

This combination converts repetitive or static fields (such as identical source IPs, ports, or capability masks) into lightweight 2-byte instructions, achieving a compression ratio of 3x to 5x.

4.4 Elastic Memory Management

Rather than employing a static ceiling on log capacity, Mini Guardian implements an **Elastic Memory** budget system to govern the Hot Tier:

- **LRU Ticking Order:** Each insertion into the Hot Tier registers a tick counter. Ticks are logged in an LRU queue (`lru_ticks`).
- **Proactive Eviction:** If the current hot buffer size (computed as `entries × 64 bytes`) surpasses the configured budget limits, the elastic manager evicts the coldest signatures from the front of the `VecDeque`, compressing and appending them to the Warm Tier.
- **Dynamic Capacity adjustment:** This allows the system to scale its in-memory footprints down when budget limits change at runtime.

4.5 Impossible State Invariants

Enforces state rules at the boundary:

- **NeverWithCapability:** Blocks a component from executing if loaded with an unauthorized cap token.
- **NeverCoexist:** Prevents two conflicting components from running concurrently.
- **NeverModified:** Alerts and rolls back changes to boot-time files.

5. Developer Workflow Compatibility

A common failure mode of desktop security agents is generating excessive false positives against legitimate developer tooling — IDEs, package managers, build systems, compilers, and language servers routinely exhibit behaviors that malware detection engines are trained to flag. Mini Guardian addresses this through a three-layer strategy of trust expansion, relaxed thresholds, and automatic detection.

5.1 Expanded Trust Origin

The trust origin module (`trust_origin.rs`) walks the `/proc/PID/status` PPid chain to classify processes as user-launched, system services, browser children, or unknown. Processes with trusted ancestry are monitored but never frozen or killed. The original trust base covered only GNOME, COSMIC, systemd, and browser processes — excluding the entire developer toolchain.

Mini Guardian v0.4 extends `TRUSTED_DESKTOP_COMMS` to include terminal emulators (`alacritty`, `kitty`, `wezterm`, `foot`, `ptysis`, `konsole`), IDEs and editors (`code`, `cursor`, `zed`, JetBrains products, `emacs`, `sublime_text`, `nvim`), build systems (`make`, `ninja`, `cmake`, `cargo`, `rustc`, `gcc`, `clang`), package managers (`npm`, `yarn`, `pnpm`, `pip`, `python`, `go`), and developer utilities (`git`, `docker`, `kubectrl`, `rg`, `fzf`). This ensures any process spawned by these tools inherits user-launched trust through ancestry, preventing auto-kills during compilation, package installation, or IDE language-server startup.

Similarly, the anonymous memory whitelist (`ANON_MEMORY_WHITELIST`), injection exclusion list (`default_injection_excluded`), and process exclusion list (`default_excluded_processes`) are updated to include all major IDEs, language servers (`rust-analyzer`, `clangd`, `gopls`, `pylsp`, `pyright`),

package managers (yarn, pnpm, npx, pip3, pipx), and container orchestration tools (kubectl, helm, minikube).

5.2 Development Mode

Despite trust-origin protection, behavioral anomaly detectors can still trigger on developer activity: parallel builds produce execve bursts of 50+ processes per second, `npm install` writes tens of thousands of files, and first-time binary execution triggers novel-binary alerts. Mini Guardian v0.4 introduces a configurable **development mode** (`dev_mode`, `DevModeConfig`) that relaxes detection thresholds when development tools are active:

- **EWMA alpha reduction:** The anomaly detector's exponential weighting factor is lowered from 0.25 to 0.15 for `execve` and `file_mod` signals. This slows baseline adaptation, smoothing out build-induced bursts.
- **Z-score threshold elevation:** The anomaly threshold is raised from 2.5 to 3.5, and the critical threshold from 4.0 to 5.0, requiring greater behavioral deviation before alerting.
- **Rapid-fire threshold increase:** The per-second `execve` burst limit is raised from 10 to 50, accommodating parallel compilation (`make -j8`, `cargo build`, `tsc --watch`).
- **Novel binary suppression:** First-time execution alerts are disabled, so compiling and running a new binary for the first time does not trigger `NovelBinary`.

These parameters are configurable in `miniguard.toml` under the `[dev_mode]` section:

```
[dev_mode]
enabled = false          # manual override
auto_detect = true       # scan /proc for dev tools automatically
ewma_alpha = 0.15
z_threshold = 3.5
rapid_fire_threshold = 50
ignore_novel_binaries = true
```

5.3 Automatic Detection

When `auto_detect = true` (the default), Mini Guardian scans `/proc` every six ticks (30 seconds) for known development tool process names. If any are found, the relaxed thresholds are applied automatically; when all dev tools exit, the system restores the original detection parameters. A `--dev-mode` CLI flag provides manual override for headless or CI environments where auto-detection may be impractical.

This design ensures that Mini Guardian maintains its full detection rigor during normal desktop use while seamlessly accommodating the anomalous-but-legitimate activity patterns of software development.

6. Advanced Constitutional & Algorithmic Security

Mini Guardian incorporates multiple layers of algorithmic containment and verification derived from the Jayce Kernel substrate. These engines operate continuously to analyze network flows, predict software failures, enforce structural constraints, and detect state tampering.

6.1 DDoS Detection & Void Pit Routing

Multi-node service networks and edge controllers are highly vulnerable to distributed denial-of-service (DDoS) and injection floods. Mini Guardian utilizes the **Network Shield** (`shroud_network.rs`) classifier and containment loop:

- **Telemetry-Based Classification:** Connections are evaluated against active counts of boundary violations and capability denials. The engine ranks threat severities across five levels (None, Probe, UnderAttack, Sustained, and Quarantine).
- **Void Pit Routing:** Once a source IP is classified as Sustained or Quarantine, the system initiates **Void Pit containment**. Instead of simply dropping packets, the traffic is isolated into a virtual “quarantine universe” where the sender receives fabricated or delayed responses. This absorbs attacker bandwidth and disrupts target exploration while completely shielding the core host systems.

6.2 Predictive Execution Horizon (PEH)

Traditional monitoring reacts **after** a vulnerability is exploited. Mini Guardian’s **Predictive Execution Horizon** (`predictive_horizon.rs`) introduces linear state forecasting:

- **Linear Delta Extrapolation:** The PEH tracks consecutive telemetry snapshots (monitoring capability denials, starvation events, latency overruns, and scheduling jitter).
- **Hazard Forecasting:** Every tick, the engine projects system state 8 to 20 ticks into the future. It evaluates threat horizons for key risks including runaway execution cycles, scheduling deadlocks, timing anomalies, and capability deviations.
- **Proactive Mitigation:** If a hazard is predicted, the scheduler triggers early containment actions (such as schedule adjustments or cap-masking) before the exploit manifests.

6.3 Impossible State Detection (ISD)

Adversaries targeting host processes often attempt to spoof system libraries, patch memory regions, or rollback counters. The **Impossible State Detector** (`impossible_state.rs`) enforces mathematical consistency checks:

- **Quantile Timing Inversion:** Verifies that lower-quantile latencies never exceed higher-quantile limits (e.g., $p_{50} > p_{95}$). Violations indicate clock tampering or artificial delay injections.
- **Counter Regression Check:** Ensures monotonically increasing telemetry metrics (such as connection counts, exec counts, and elapsed ticks) never decrement, blocking rollback exploits.
- **Accounting Integrity:** Ensures that aggregate counts (allowed + denied + quarantined) perfectly sum to raw arrivals (arrivals), flagging state manipulation or message drops.
- **Flow Contradiction:** Detects structural logic conflicts (e.g., `strict_mode` is enabled while `network_enabled` is active).

If any mathematical impossibility is hit, the system triggers a `FreezeAndContain` or `QuarantineAndRecover` action.

6.4 Constitutional Shadow Layer (CSL)

To guarantee that Mini Guardian cannot be repurposed or manipulated by hostile software, it implements the **Constitutional Shadow Layer** (`constitutional_shadow.rs`). It enforces six hardcoded, non-bypassable constraints:

- `CONSTITUTIONAL_NO_MASS_SURVEILLANCE`: Rejects any attempt to enable bulk telemetry or covert user recording.
- `CONSTITUTIONAL_NO_DECEPTIVE_PATTERNS`: Blocks user-interface spoofing or deceptive choice layouts.
- `CONSTITUTIONAL_VOLUNTARY_PARTICIPATION`: Ensures operators maintain explicit control over active components.
- `CONSTITUTIONAL_FAIL_CLOSED`: Directs execution flow to freeze or drop permissions if validation errors occur.

- `CONSTITUTIONAL_LOCAL_ONLY`: Structurally denies any cloud-based telemetry or external phone-home connections.
- `CONSTITUTIONAL_USER_VISIBLE`: Mandates that all containment actions are visible to the host operator.

7. Defense of the Daemon Itself

Mini Guardian’s threat model includes the advanced attacker who attacks the security daemon itself rather than the workloads it protects. A root attacker can debug, trace, disassemble, or freeze the daemon so it cannot defend the host. Mini Guardian meets this on three fronts, none of which depend on code running **inside** the daemon (anything inside the daemon is compromised along with it).

7.1 Release Hardening

- **Stripped release profile**: `[profile.release]` sets `strip = "symbols"`, `lto = true`, and `codegen-units = 1`. The deployed binary exposes no function names to disassemblers (main binary 16 MB, sentinel 0.5 MB).
- **Statically linked (musl)**: the workspace builds against `x86_64-unknown-linux-musl` (link-self-contained), so both binaries are fully static — no `glibc/libc`. so dependency surface to load hijack or inspect.
- **Immutable on disk**: the deployed binaries carry the immutable attribute (`chattr +i`), preventing substitution or patching between restarts.
- **Runtime hardening**: the daemon sets `PR_SET_DUMPABLE = 0` (disables ptrace attachment and core dumps) and monitors `TracerPid`.

7.2 Ship Seal (`seal.rs` + `scripts/seal.sh`)

Beyond hardening comes the ship-seal phase that makes the deployed product as close to “damn near impossible to reverse or tamper” as userspace allows:

- **String obfuscation** (`obfstr`): capability-revealing strings — the sealed hash manifest, the control-socket path, watchdog and sentinel heartbeat and freeze-log paths, and the daemon/sentinel binary paths — exist on disk only as decrypt-at-runtime buffers. strings over either binary no longer reveals the watch/restart architecture. Paths that are trivially discoverable on the live filesystem anyway (policy lists, const contexts) are left plain as an accepted residual.
- **argv/envIRON scrub**: `scrub_argv_envIRON` zeroes `/proc/self/cmdline` and `/proc/self/envIRON` at startup, closing the process-masquerade tell.
- **Tracing is lethal**: a foreign `TracerPid` is no longer merely escalated — the process self-SIGKILLs for a clean `systemd` restart, so the attacker loses the traced session outright.
- **Sealed-hash cross-verification**: `scripts/seal.sh` pins SHA-256 of both deployed binaries in `/etc/miniguard/seal.hashes` (immutable, `chattr +i`). The daemon re-verifies every 120 s and the sentinel every 60 polls; on mismatch the daemon escalates to `paranoid + audit` (severity 3, truth-chain verify, forensic) and the sentinel SIGKILLs the drifted daemon.
- **Read-only deployment**: `scripts/seal.sh --mount-ro` bind-mounts `/usr/local/bin` read-only as a second tamper layer beyond `chattr`.
- **Host attestation surfaced**: the `seal` socket command, the forensic `seal` section, and the TUI `Seal` line report `kernel_lockdown` (`/sys/kernel/security/lockdown`), Secure Boot (efivarfs), TPM presence, and on-disk immutability of both binaries — an incident reviewer sees whether the host boundary behind the daemon is itself sealed.
- **Observability is socket-only**: the `seal_logger` in-memory ring replaces `journald` (`journalctl` shows nothing from either process). The `logs` socket command returns the last 400 ring entries.

- **Documented-future:** LLVM obfuscation passes (requires a rustc toolchain fork; the obfuscator crate is not published) and eBPF program-name randomization (aya 0.14 names come from the embedded .o; visible only to root via debugfs) are accepted residuals.

7.3 Anti-Analysis Detection (`analysis_detector.rs`)

Reverse-engineering is observable, and Mini Guardian watches for it. Every tick it scans `/proc` for analysis tooling **targeting the daemon**:

- **Debuggers/tracers:** `gdb`, `lldb`, `strace`, `ltrace` — always suspicious.
- **Dynamic instrumentation:** `frida` — always suspicious.
- **Disassemblers:** `radare2`, `IDA`, `Ghidra`, `Binary Ninja`.
- **Binary inspection utilities:** `objdump`, `readelf`, `nm`, `strings`.
- **Memory readers:** any process opening `/proc/<daemon_pid>/mem` or `maps` (a root attacker bypasses `PR_SET_DUMPABLE` — this catches the bypass).
- **Targeting signals:** process command lines referencing the daemon's PID, executable, or process name.

Severity is laddered: tool presence alone is informational; `frida`, targeted analysis, or a memory read escalates to attack memory (learning), the illusion layer (deception), and paranoid mode.

7.4 Antifreeze Sentinel (`sentinel.rs` + `miniguard-sentinel`)

A root attacker can `SIGSTOP` the daemon — it stops defending the host, and anything running inside it stops too. Antifreeze therefore lives in a **separate** systemd-managed process (`miniguard-sentinel`, itself stripped/immutable, `PR_SET_DUMPABLE = 0`):

- Every 2 s it reads `/proc/<daemon>/status` (found by exe path, so the daemon's kernel-thread comm masquerade does not blind it) plus the daemon's heartbeat file.
- Freeze detection: State: `T` (stopped), or a heartbeat stale beyond 45 s (the daemon writes its heartbeat from a task independent of the tick loop, so a frozen daemon cannot fake freshness).
- Response ladder: first freeze → `SIGCONT` + evidence capture; ≥ 3 freezes within 60 s → `SIGKILL` for a clean systemd restart.
- Evidence: each freeze writes a `FREEZE_EVENT` line (timestamp, PID, detection method, freeze count, root-owned culprit processes) to a log the daemon tail-consumes on resume — it escalates the event into the brain (severity 3), metrics, truth-chain verification, illusion probes, and paranoid mode.
- Mutual watch: the daemon restarts a dead sentinel (heartbeat stale > 90 s), so killing the sentinel is not a free win.
- Seal cross-verification: the sentinel also verifies the daemon binary against `/etc/miniguard/seal`.hashes every 60 polls and `SIGKILLS` a drifted daemon, and the daemon does the reciprocal check every 120 s — a swapped binary is detected and rolled back by systemd restart within minutes.

7.5 Shadow Backup and Auto-Rollback (`shadow_backup.rs`)

Mini Guardian keeps baseline shadow copies of security-critical files (`/etc/passwd`, `/etc/shadow`, `/etc/group`, `/etc/gshadow`, `/etc/sudoers`, `/etc/ssh/ssh_config`, `/root/.ssh/authorized_keys`, `/etc/crontab`) under `/var/lib/jayce/miniguard/shadow/` and auto-restores them on tamper. `/etc/ld.so.preload` is forbidden-on-sight (deleted). MiniGuardian-style, the attacker's version is preserved in `shadow/evidence/` **before** restore, its content hash feeds attack memory, and the restore anchors the truth chain. A shadow file that is itself tampered refuses to restore; an administrator re-baselines by deleting the shadow file for a path.

7.6 Container Awareness (`container_monitor.rs`)

Mini Guardian scans `/proc` every 30 s for the container threat surface: runtime detection (dockerd/containerd/crio/kubelet/podman), container inventory from cgroups, docker.sock mounts, CAP_SYS_ADMIN privileged containers, host-namespace sharing, sensitive host mounts (`/etc`, `/root`, `/proc`, `/sys`, `/dev`, `/`), and escape paths (`/proc/1/mem`, `release_agent`, `/etc/shadow`).

7.7 Forensic Report

A forensic socket command emits a full incident-review JSON document: audit chain, kernel truth chain, void-pit roster, attack memory, playbook state, deception observations, artifact inventory, and seal posture (kernel lockdown / Secure Boot / TPM / binary immutability / manifest presence) — a MiniGuardian-style review artifact rather than a cloud upload.

8. Fleet Visibility (Cosmic Map)

Mini Guardian packages a fleet observatory into the same binary (`cosmic_map/` vendored crate). `miniguard --map` opens the Observatory window; the root daemon writes JAYCE_TLM: telemetry frames (this host plus every gossip peer, threat-colored) to a pipe read by the unprivileged UI — the fleet view never requires root.

9. Gossip and Cluster P2P Integration

To defend multi-node infrastructures (such as clusters, server pools, or robotic fleets), Mini Guardian integrates a decentralized peer-to-peer gossip protocol. This allows individual cohost nodes to share security posture metrics, coordinate firewall rules, and relay system signals in real-time.

9.1 Mutual Authentication and Sidechannel Mitigation

The gossip connection handshake begins with a strict mutual authentication protocol:

- **Cleartext Handshake:** The connecting node transmits its auth token, which the receiver compares in constant-time (`ct_eq_bytes`) against its local credential. This mitigates side-channel timing analysis attacks targeting token structure.
- **Mutual Verification:** If valid, the receiver transmits its own auth token back to complete the mutual trust handshake before entering the telemetry loop.

9.2 Gossip Message Architecture & Anti-Replay

Nodes exchange structured JSON packets containing local threat metrics, sequence counters (`seq`), and active firewall blocks. The protocol enforces integrity through two cryptographic mechanisms:

- **HMAC-SHA256 Signatures:** An HMAC signature is calculated over the canonical JSON body of the message using the shared authentication token, protecting against data tampering or injection.
- **Hash-Chained Integrity:** Each message includes a `chain_hash` linked to previous message hashes. Receivers cross-reference incoming hashes against historical states to reject replay or out-of-order packet injections.

9.3 Rate Limiting and Memory Bounding

To prevent denial of service (DoS) of the security daemon, Mini Guardian applies:

- **Token Bucket Rate Limiting:** Restricts peer message throughput on a per-IP basis.
- **Oversized Packet Rejection:** Lines exceeding standard limits are dropped, and memory-allocated buckets are actively pruned to keep memory footprints bounded.

9.4 Cryptographic Transport (mTLS)

Gossip communication supports mutual TLS (mTLS) built on `rustls`:

- **Enterprise Deployment:** Restricts communication to nodes presenting valid certificates signed by a trusted Enterprise CA.
- **Developer Mode (Auto-TLS):** If certificates are not configured, the daemon generates ephemeral self-signed CA and node certificates at startup, enabling encrypted cluster communication out of the box.

10. Verification and Assessment

10.1 Test Suites

Mini Guardian is verified through three independent test suites:

Suite	Tests	Status
miniguard lib	805	PASS (0 failures)
miniguard integration suites	119	PASS (0 failures)
jayce_kernel	83	PASS (0 failures)
trust_core	27	PASS (0 failures)
trust_terminal	17	PASS (0 failures)
Total	1051	All pass

Table 2: Test suite results

10.2 Competitive Security Assessment

Mini Guardian includes a built-in competitive assessment (`miniguard-status assessment, assessment.rs`) that scores its own security posture across multiple dimensions and compares against CrowdStrike Falcon. Scores are self-assessed based on implemented capabilities, verified via test coverage and code audit:

Dimension	Mini Guardian	CrowdStrike Falcon
Enforcement	9/10	8/10
Detection	10/10	9/10
Real-time	9/10	7/10
Scalability	7/10	8/10
Privacy	10/10	4/10
Overall	8.0/10	6.9/10

Table 3: Competitive security assessment

Scoring rationale:

- **Enforcement (9):** eBPF/XDP NIC-level filtering, Landlock filesystem sandboxing, chained seccomp-BPF, capability-gated containment with 5 Brain states
- **Detection (10):** EWMA anomaly detection (8 metrics/process), cross-component correlation, PEH hazard projection (8-20 tick horizon), ISD mathematical invariant checks, Prompt Guard (24 families, 200+ patterns), 5-layer Memory Poisoning Defense

- **Real-time (9):** Deterministic 1,000 Hz tick loop with fixed budgets; preprocessing and scan pipeline in tick loop averages approximately 164ms per 5s cycle during normal desktop operation with 4% sustained CPU
- **Scalability (7):** Gossip cluster P2P mesh with mTLS, HMAC-SHA256, hash-chained anti-replay, up to 16 peers, health metric propagation, K8s readiness/liveness probes; no centralized coordinator required; not yet tested at 100+ node scale
- **Privacy (10):** All analytics local-only; no cloud telemetry, no phone-home, no external ML pipeline; optional gossip sharing is explicitly opt-in with cryptographic authentication

11. Conclusion

Mini Guardian provides a robust userspace containment layer that integrates eBPF network filtering, Landlock filesystem sandboxing, and chained seccomp-BPF filters, deployed natively as a systemd service with no container runtime or kernel modifications required. The autonomous Brain with 5 feedback loops enables local-only threat learning, while the gossip cluster extends protection to multi-node deployments and the packaged Cosmic Map observatory renders the fleet. The daemon defends itself against advanced attackers through release hardening, string-obfuscated paths, anti-analysis detection, and an out-of-process antifreeze sentinel, and the ship-seal phase binds the whole deployment together: statically linked binaries, sealed-hash cross-verification between daemon and sentinel, immutable and read-only on-disk deployment, and surfaced host attestation. With 1051 passing tests and a self-assessed security score of 8.0/10, Mini Guardian demonstrates that comprehensive, non-intrusive security can be achieved entirely in userspace.